Natural Language Processing

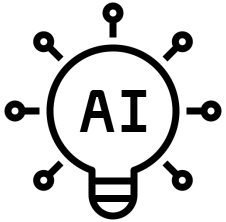
Session 10 – Working with LLMs

Prof. Dr. Richard Sieg
SoSe 2026

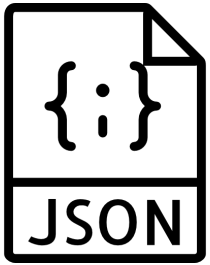
Schedule

Date	Weekday	CW	Room	Topic
16.04.	Thursday	16	149	Introduction: The World of NLP
23.04.	Thursday	17	149	Text Processing Fundamentals
30.04.	Thursday	18	149	Linguistic Annotations & Pattern Matching
05.05.	Tuesday	19	154	Text Vectorization: From Text to Numbers ⚠️ Substitute for 14.05.
07.05.	Thursday	19	149	Text Classification
28.05.	Thursday	22	149	Language Models, Word Vectors + 🎤 Guest Lecture Ines Montani
01.06.	Monday	23	390	📝 **Exams — Master DS Students**
02.06.	Tuesday	23	154	BERT: Pre-Training & Fine-Tuning ⚠️ Substitute for 04.06.
11.06.	Thursday	24	149	The BERT Ecosystem
18.06.	Thursday	25	149	Introduction to LLMs
25.06.	Thursday	26	149	Working with LLMs
02.07.	Thursday	27	149	Ethics, Limitations & Exam Preparation
09.07.	Thursday	28	149	📝 Exams (afternoon) — Bachelor DIS Students
10.07.	Friday	28	149	📝 Exams (afternoon) — Bachelor DIS Students

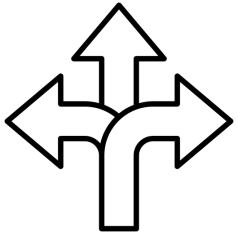
Today we will learn...



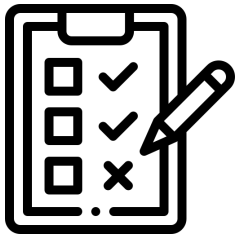
how models reason



how to get usable output



which tool fits which task



how to evaluate LLMs

Agenda

01. Reasoning Models

02. LLMs in Practice

03. Evaluating LLMs

04. Outlook

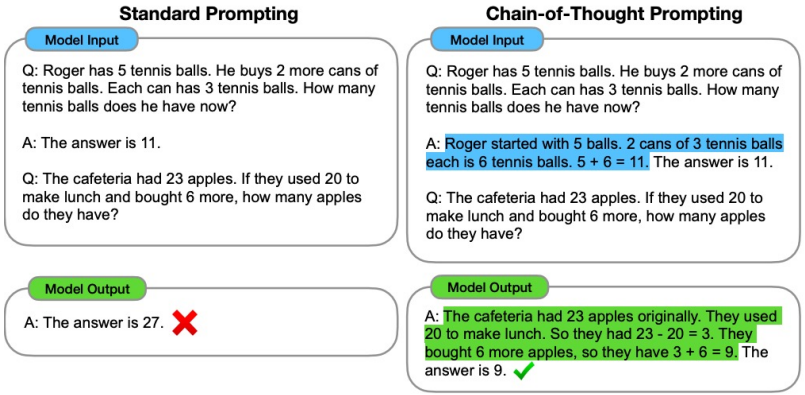
01.

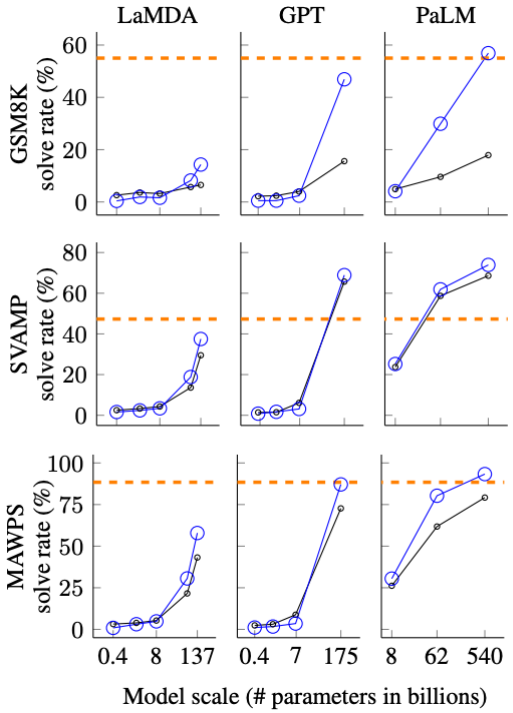
Reasoning Models

Recap: Chain of Thought (CoT) Prompting

- Idea: make the model show its work before answering.

  On multi-step problems this helps a lot.
- Zero-shot CoT: just add "Let's think step by step."
- Few-shot CoT: include worked examples that show the reasoning, not just the answer
- Self-consistency: sample several reasoning paths and take the majority answer. Aggregating beats any single path

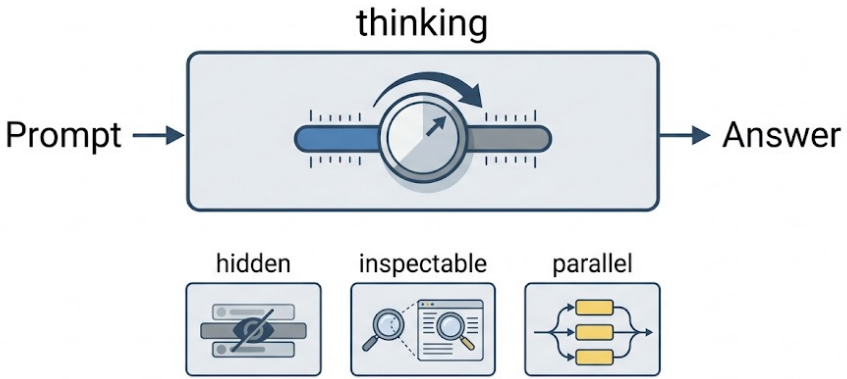




Chain-of-Thought Prompting Elicits Reasoning in Large Language Models [Wei et al, 2022]

Reasoning Models

- Train the model to deliberate before answering, instead of prompting for steps.
- Test-time compute: model spends extra computation at answer time.
  More thinking → better answers on hard problems
- By mid-2026 the default, not a niche: most frontier models think first, deciding per query (e.g. OpenAI o-series, DeepSeek-R1). Reasoning = a mode, not a model class
- Where the thinking lives varies: hidden tokens, an inspectable budgeted block, or parallel hypotheses



Test-Time Compute: a Third Way to Scale

- The last session scaled at training time. Reasoning adds a lever at answer time.
- Train time: model size + data + compute (the scaling laws)
- Answer time: let the model think longer on the question at hand
- 2026 slogan: more thoughtful, not just bigger
- Tradeoff: can beat a larger ordinary model on hard problems, but slower and pricier per answer -> you pay for thinking tokens

Prompting Reasoning Models

A reasoning model already thinks -> prompt it differently.

- Do not hand-hold: drop "let's think step by step". Give a clear brief + the goal, let it plan
- Control the effort: higher thinking budget for hard problems, lower for easy ones
- Use for: hard, multi-step work -> math, logic, planning, tricky extraction
- Skip for: simple lookups, classification, formatting -> a standard model is faster + cheaper

Good at	Bad at
+ Deductive or inductive reasoning (e.g., riddles, math proofs)	– Fast and cheap responses (more inference time)
+ Chain-of-thought reasoning (breaking down multi-step problems)	– Knowledge-based tasks (hallucination)
+ Complex decision-making tasks	– Simple tasks (“overthinking”)
+ Better generalization to novel problems	

Sebastian Raschka: Understanding Reasoning LLMs

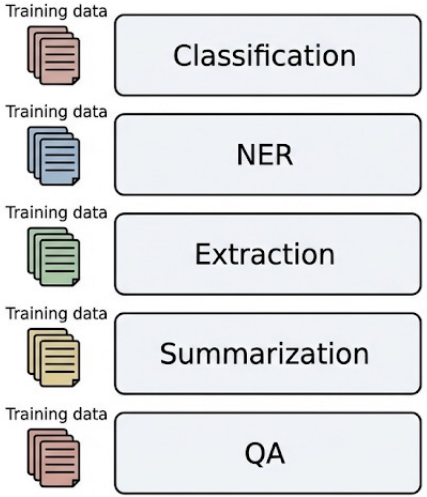
02.

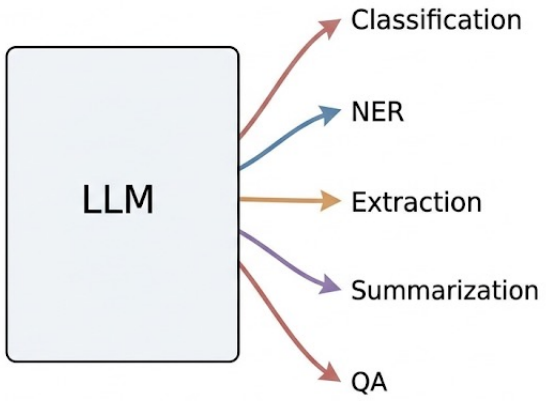
LLMs in Practice

One Model, Every Task

- Classification, NER, extraction, summarization, QA: all just instructions, no task-specific training set
- Brings world knowledge the old pipelines lacked: knows ”hyperpop" with zero examples
- The catch: a classifier returns a clean label, an LLM returns a sentence, and a pipeline cannot use a sentence

The "one model does it all" shift: a model per task before, one instructed model now





How Good Is It, Really?

It depends on the task 🤷

- Benchmarks: MMLU (knowledge), GSM8K (math), HumanEval (code).
  Frontier LLMs score high and climb fast (see Evaluation Section)
- Discriminative tasks with labels: a fine-tuned BERT is often as good or better than a few-shot LLM, and far cheaper.
- Why: few-shot mostly teaches task + format, not new answers →no real training signal
- So "LLM" does not always mean "better". It means better when you lack labels or need flexibility and world knowledge

Not everything is LLMs. Use the smallest model that achieves the desired performance.

Where LLMs Genuinely Win

- Generative Tasks: Open-ended generation, no single right answer
- Generative seq2seq: summarization, text simplification, paraphrase, style transfer, translation, grammar correction, data augmentation
- Hard before: free-text output, many valid answers -> cannot frame as a label, needs fluent generation. Was encoder-decoder territory (T5, BART)
- What changed: one instructed decoder does them zero-shot → no model per task. NLP expands from understanding to generating
- On our data: summarize a song, simplify dense lyrics, restyle a song (the tutorial), paraphrase a chorus

Structured Output

- Output is for a program, not a person → treat it as data that is valid every time.
- Define a schema, demand it:
  `{"genre": "...", "confidence": 0.0-1.0}, "return only JSON, no prose"`
- Parse + validate: keys present, values in the allowed set, retry once on malformed output (there are Python packages for that)
- Enforce at the API level: Gemini **`response_schema + response_mime_type`**; Mistral / OpenAI **`response_format`** → model constrained to valid JSON
- Tag your sections: **`### LYRICS`** or **`<lyrics> ... </lyrics>`** → instruction and data never blur

Batching

- One request with many documents in → a JSON list out.
- Key every result by id, never trust order: the model can drop or reorder items. Match by id, re-request the misses
- Payoff: far fewer round trips → lower cost and latency
- The tradeoff: too many items per batch → per-item accuracy slips (attention spread thinner). A real batch-size vs accuracy vs cost curve, so pick a size and check it

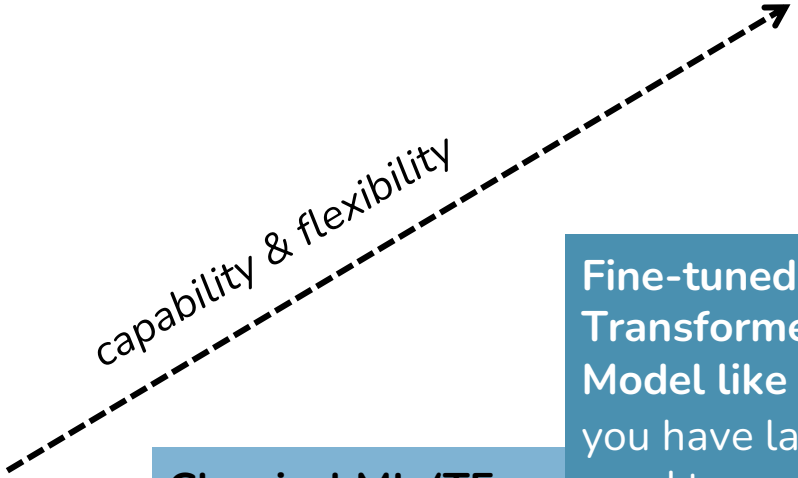
The Costs That Matter

- Benchmark often focus on accuracy and score and ignore other costs.
- Size: fine-tuned BERT ~110M vs frontier LLM hundreds of billions to trillions: orders of magnitude in memory, hardware, and who can run it
- Money: an API LLM bills per token, forever; a small model trains once and serves cheap.
- Environment: training and serving burn real energy and carbon, and bigger means more
- Interpretability: TF-IDF weights are inspectable and auditable, an LLM is an opaque black box. Matters for debugging, trust, regulation

A responsible choice weighs size, money, carbon, and interpretability, not accuracy alone.
Often the "boring" model is the professional one.

More on that in our ethics session

When to Use What



Rules / regex
the pattern is fixed and obvious

Classical ML (TF-IDF, Naive Bayes)
you have labels, want speed and transparency

Fine-tuned Transformer Model like BERT
you have labels, need top accuracy at low latency

Prompted LLM
little or no labeled data, or open-ended and needs world knowledge

Prompt Engineering or Fine-Tuning?

Prompt Engineering

Tell the model what to do at runtime, every time
"You are an expert in..."

Benefits

- Adaptable – for many inputs
- Instant – no training
- Flexible – change anytime
- Experimenting is cheap

+ RAG
Give access to current knowledge

👉 **You should start here**

- Inefficient
- Worse performance than fine-tuning
- Sensitivity to wording
- Lack of clarity

Fine-Tuning

Modify model weights through training examples
Permanent behavior change

Only when

- Fixed & specific task
- A smaller model is needed:
- Lower latency
- Smaller costs

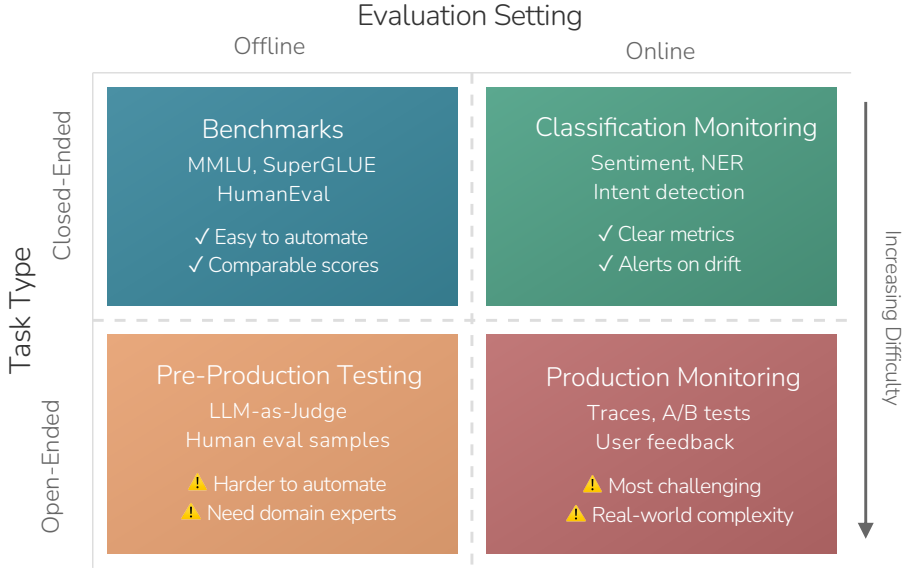
👉 **Only in rare cases**

03.

Evaluating LLMs

The Evaluation Challenge

- Two major evaluation paradigms
- ***Discriminative***: Closed-ended evaluations
  - Classification, QA, extraction etc
  - Limited correct answers
  - Automatic evaluation possible
- ***Generative***: Open-ended evaluations
  - Generation, conversations
  - No reliable way to evaluate
  - Many valid answers
  - Evaluation is inherently difficult



High benchmark scores don't guarantee real-world performance. Models can exploit spurious correlations without truly understanding the task.

Two Evaluation Worlds

Benchmark-Evals
(Is the model good in general?)

- Multiple-choice knowledge (MMLU)
- Verifier: prüfbare Aufgaben (Mathe, Code)
- Verifiable tasks: math, code
- Leaderboards, models head to head

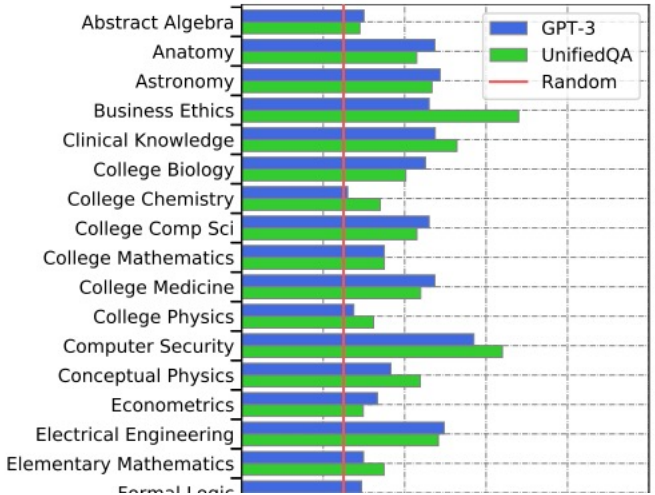
Benchmark evals choose the model; product evals judge the application

Product-Evals
(Does my application work?)

- Is the generated song really in the artist's voice?
- Does every response come back as valid JSON?
- Does the chatbot stay on topic and follow the system prompt?
- Does it handle weird or adversarial inputs safely?

Multitask Benchmarks - MMLU

- Massive Multitask Language Understanding
- Contains 57 diverse knowledge tasks
- Was/Is the “go to” LLM benchmark (up until shortly)
- Benchmarks can’t keep up with LLM development



Why isn’t there a planet where the asteroid belt is located?
(A) A planet once formed here but it was broken apart by a catastrophic collision.
(B) There was not enough material in this part of the solar nebula to form a planet.
(C) There was too much rocky material to form a terrestrial planet but not enough gaseous material to form a jovian planet.
(D) Resonance with Jupiter prevented material from collecting together to form a planet.

Figure 16: An Astronomy example.

If each of the following meals provides the same number of calories, which meal requires the most land to produce the food?
(A) Red beans and rice
(B) Steak and a baked potato
(C) Corn tortilla and refried beans
(D) Lentil soup and brown bread

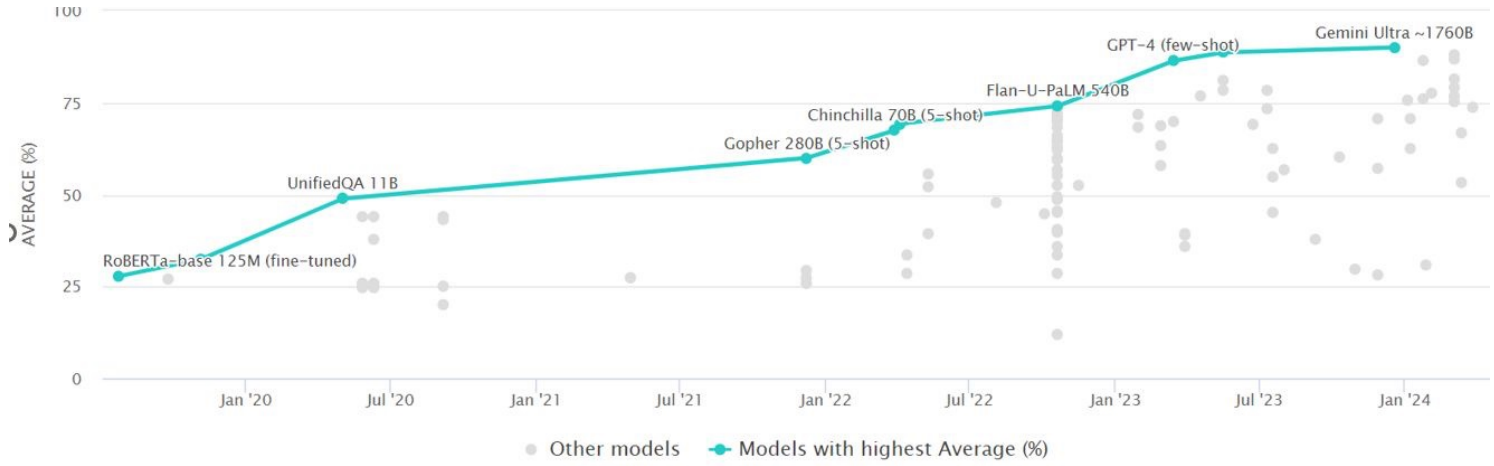
Measuring Massive Multitask Language Understanding [Hendrycks et al, 2020]

MMLU progress

GPT3 was SOTA

Model	Humanities	Social Science	STEM	Other	Average
Random Baseline	25.0	25.0	25.0	25.0	25.0
RoBERTa	27.9	28.8	27.0	27.7	27.9
ALBERT	27.2	25.7	27.7	27.9	27.1
GPT-2	32.8	33.3	30.2	33.1	32.4
UnifiedQA	45.6	56.6	40.2	54.6	48.9
GPT-3 Small (few-shot)	24.4	30.9	26.0	24.1	25.9
GPT-3 Medium (few-shot)	26.1	21.6	25.6	25.5	24.9
GPT-3 Large (few-shot)	27.1	25.6	24.3	26.5	26.0
GPT-3 X-Large (few-shot)	40.8	50.4	36.7	48.8	43.9

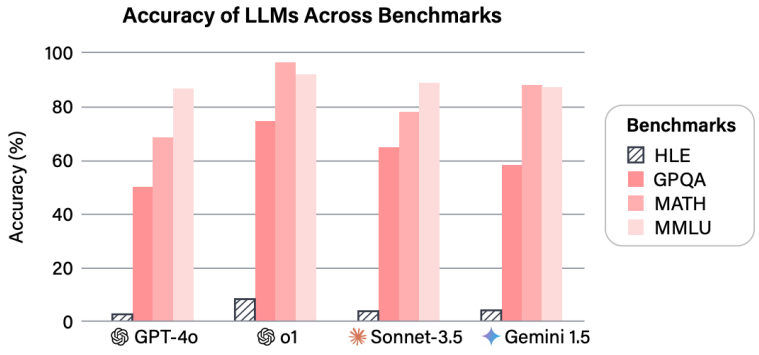
LLM Stats Leaderboard

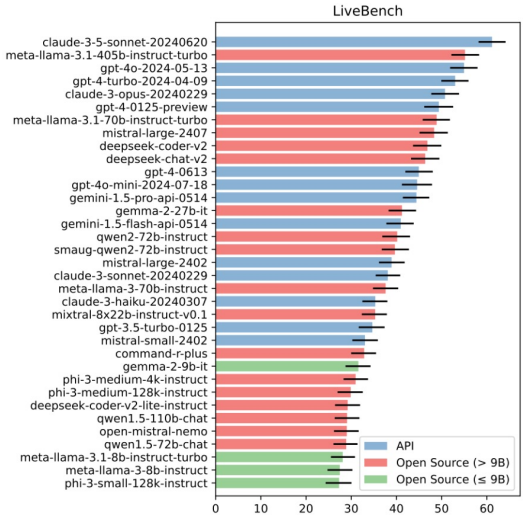


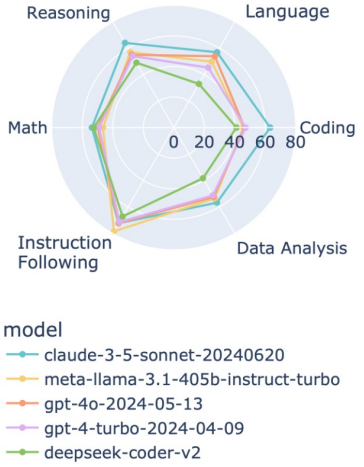
Measuring Massive Multitask Language Understanding [Hendrycks et al, 2020]

Some Other Benchmarks

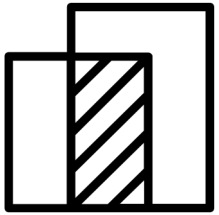
- BIG-Bench (Google, 2022) contains 204 tasks
- Humanity's Last Exam (AI Safety, 2025)
- LiveBench (2024) gets updated monthly, to limit potential contamination into the training data of LLMs







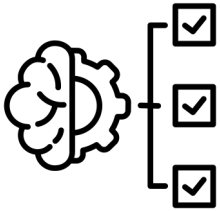
Scoring Generated Text



Content Overlap Metrics
(BLEU, ROUGE etc)

- Measure n-gram overlap with reference text
- Fast, cheap, and widely used
- Problem: no semantic understanding

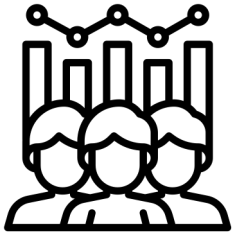
Needs Reference Text



Model Based Metrics
(BERTScore, BLEURT etc)

- Use embeddings for semantic similarity
- Better correlation with human judgement

Needs Reference Text

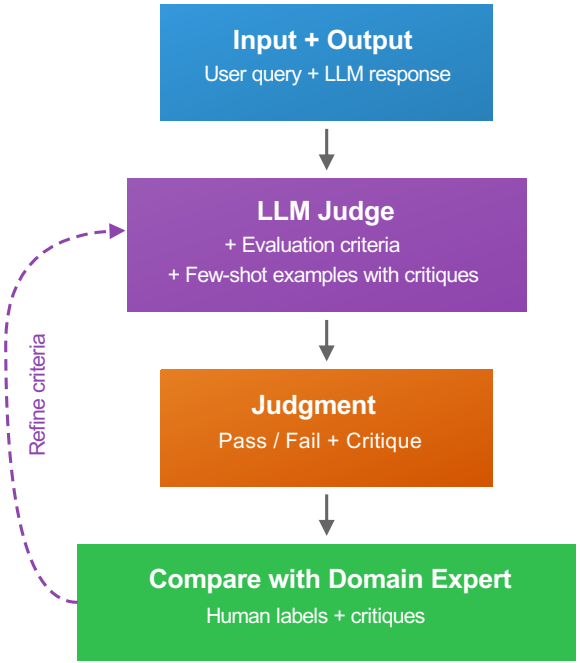


Human Evaluation

- Gold standard
- Expensive and slow

LLM as a Judge

- Issue: Human annotation does not scale
- ***LLM as a Judge***
  - LLM receives input, output, and evaluation criteria
  - LLM produces judgement and explanation (critique)



Common Pitfalls

- Position bias (prefers first answer)
- Length bias (prefers longer answer)
- Self-preference (LLM prefers own output)

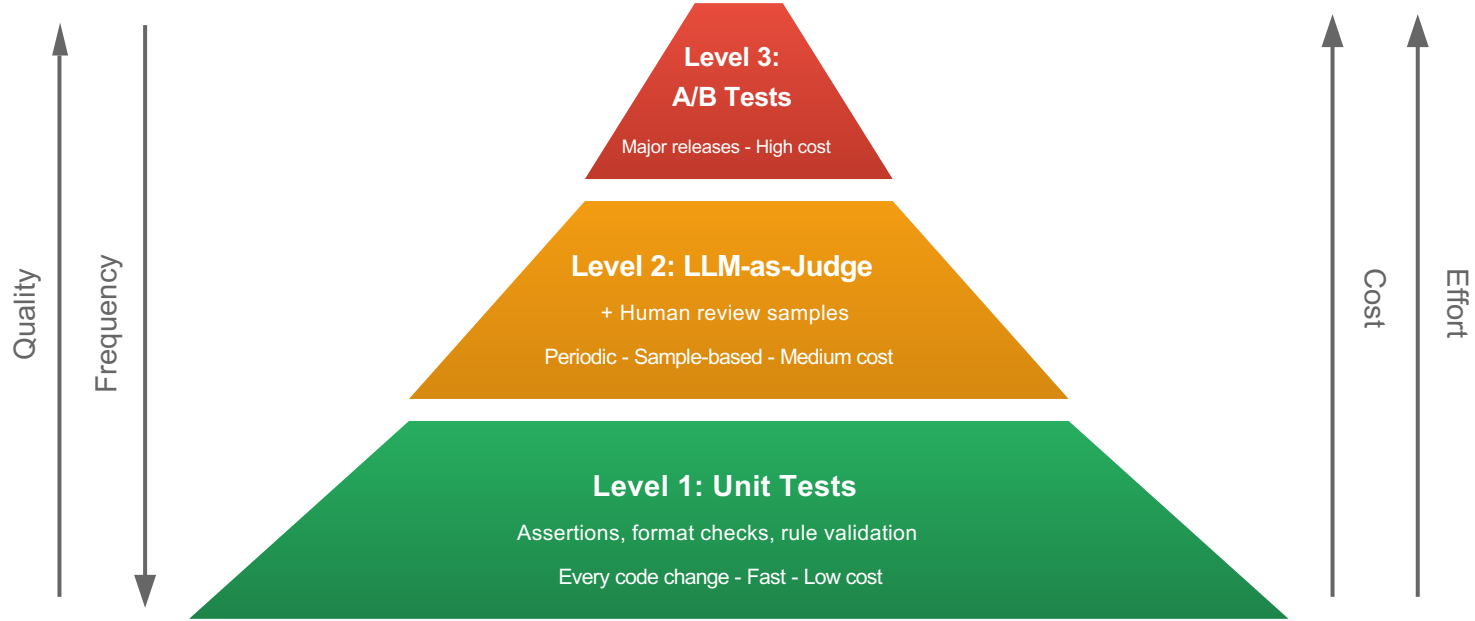
Best Practices

- Binary pass/fail decision
- Include critique
- Use task-specific criteria

Production – Traces & Monitoring

- Logging & Traces
  - Input, output, latency, costs, prompts
  - Tools: Langfuse, Langsmith, many more

🏆 **Rule Number 1**
Always look at your traces



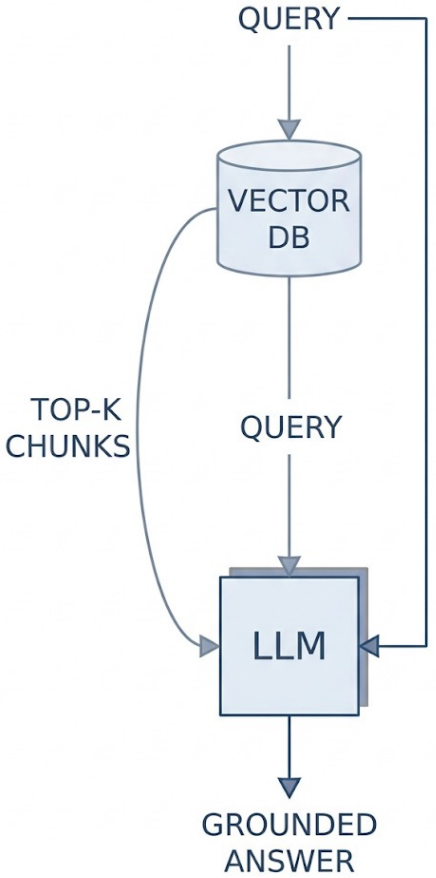
04.

Outlook

RAG: Giving the Model Knowledge

The model knows only its training data.
RAG gives it custom knowledge at answer time.

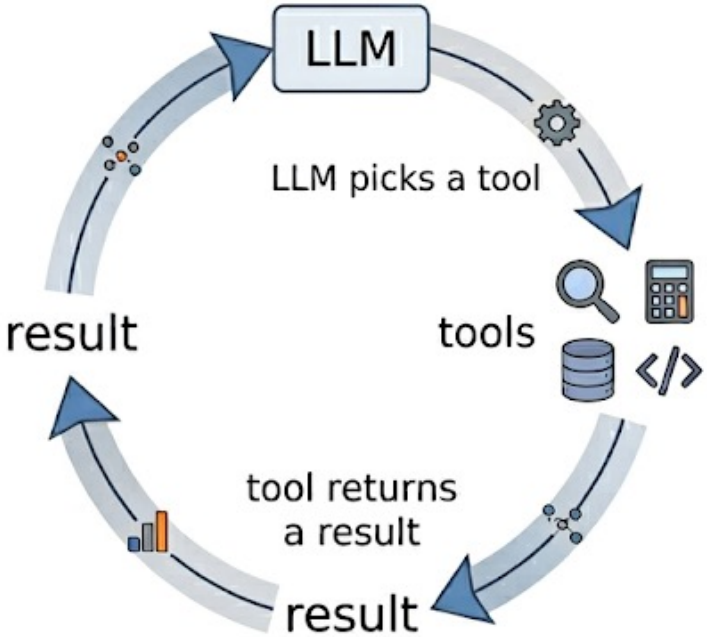
- Based on the user input, relevant documents are ***Retrieved*** from a database
  (embeddings and similarity search from Session 7)
- Those documents are given to the LLM as additional context (***Augmentation***)
- The LLM then ***Generates*** the output based on the original input and retrieved documents



Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [Lewis et. al., 2021]

Beyond a Single Prompt

- ***Tool use:*** the LLM calls functions (search, a calculator, an API)
- ***Agents:*** *"an LLM that runs tools in a loop to achieve a goal"* (Simon Willison)
- ***Context engineering:*** giving the agent the right context at each step
- ***MCP (Model Context Protocol):*** a standard way to connect models to tools and data
- ***DSPy:*** stop hand-writing prompts, optimize them automatically against a metric



Covered in DS Master Modules